

JLU ORBIT

A MAJOR PROJECT REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF
THE DEGREE OF

BACHELOR OF COMPUTER APPLICATIONS

(specialisation in Full Stack Development)

SUBMITTED BY

Mradul Yadav (2023BCAH031)

Naman Sah (2023BCAH034)

Samuel Dennis David (2023BCAH046)

UNDER SUPERVISION OF

Dr. Dileep Kumar Singh

Professor & Head

JSCA, JLU Bhopal

December-2025



JAGRAN SCHOOL OF COMPUTER APPLICATIONS (JSCA)

FACULTY OF SCIENCE AND TECHNOLOGY (FAST)

JAGRAN LAKECITY UNIVERSITY, BHOPAL (M.P.)

Table of Contents

Acknowledgment
Certificate

Abstract	i
List of Tables	ii
List of Figures	iii
List of Abbreviations	iv

1.	INTRODUCTION	1
	1.1 Problem Definition	1
	1.2 Project Overview/Specifications	1
	1.3 Hardware Specification	2
	1.4 Software Specification	2
2.	RELATED WORK	3
	2.1 Existing System	3
	2.2 Proposed System	4
	2.3 Feasibility Study	6
3.	SYSTEM ANALYSIS & DESIGN	8
	3.1 Requirement Specification	8
	3.2 Data Flow Diagram & ER Diagram	9
	3.3 Design and Test Steps / Criteria	13
	3.4 Algorithms and Pseudo Code	20
	3.5 Testing Process	26
4.	RESULTS / OUTPUTS	28
5.	CONCLUSIONS / RECOMMENDATIONS	40
6.	REFERENCES	41
7.	APPENDICES(include GitHub repository link)	42

ACKNOWLEDGMENT

It is indeed a great pleasure to express our thanks and gratitude to all those who helped us during this period. This project would not have been materialised without the help from many quarters. We sincerely thank to all the persons who ever played a vital role in the successful completion of our project.

We sincerely thank all the people who co-operate and encourage us throughout the semester and make our project work successful.

We are thankful to **Dr. Kretika Tiwari** (Project Coordinator, JSCA) who has constantly remained helpful in suggesting directions and providing ue guidance throughout the project.

We also thank **Mrs. Vaishali Sharma** (Program Leader (BCA), JSCA) for providing us the confidence and helping us in the project.

Special thanks to **Dr. Dileep Kumar Singh** (Professor & Head, JSCA) for providing this platform which helped in the development of our technical skills and provided opportunity to study and work here which imparted a lot of knowledge, experience, confidence, skills, and real field awareness to us.

It is good fortune that we had support and well wishes of many. We thank all those, whose names have not appeared here but the contributions have not gone unnoticed.

Mradul Yadav
(2023BCAH031)

Naman Sah
(2023BCAH034)

Samuel Dennis David
(2023BCAH046)

CERTIFICATE

We hereby certify that the work which is being presented in the BCA Major Project Report entitled “**JLU ORBIT**”, in the partial fulfilment of the requirements for the award of the **Bachelor of Computer Applications (FSD)** is an authentic record of our own work carried out during the session **Jul-Dec, 2025 (5th semester)** under the guidance of **Dr. Dileep Kumar Singh** (Professor & Head, JSCA). The matter presented in this Project Report has not been submitted by us for the award of any other degree/diploma elsewhere.

Mradul Yadav
(2023BCAH031)

Naman Sah
(2023BCAH034)

Samuel Dennis David
(2023BCAH046)

This is to certify that the above statement made by the students is correct to the best of my knowledge.

Date:

Dr. Kretika Tiwari
Project Coordinator (BCA)
JSCA, JLU Bhopal

Dr. Dileep Kumar Singh
Project Guide
Professor & Head
JSCA, JLU Bhopal

ABSTRACT

JLU Orbit represents a significant technological intervention in the administrative and academic communication workflow of Jagran Lakecity University (JLU). Designed as a centralized, web-based blogging and announcement platform, the project seeks to eliminate the inefficiencies inherent in traditional, fragmented communication methods such as instant messaging groups, disjointed email threads, and physical notice boards. These legacy systems often result in information silos, lack of data persistence, and reduced accessibility for students and faculty alike. By establishing a unified digital ecosystem, JLU Orbit ensures that critical departmental notifications, academic resources, and event updates are consolidated into a single, authoritative, and accessible source.

Technically, the application is built upon a robust full-stack architecture, utilizing **Node.js** for a high-performance, non-blocking backend runtime and **Express.js** to manage complex routing and middleware integration. The data layer leverages **MongoDB**, a NoSQL database, to offer schema flexibility suitable for diverse content types, including rich text and multimedia assets. This document-oriented approach allows for the efficient handling of unstructured data, ensuring that posts containing images and extensive descriptions are retrieved with low latency. The system architecture strictly follows the **Model-View-Controller (MVC)** pattern, ensuring code modularity, scalability, and long-term maintainability.

Security and data integrity were paramount during the development lifecycle. The system implements a secure authentication mechanism using **JSON Web Tokens (JWT)** for stateless session management and **bcrypt** for cryptographic password hashing. This ensures that only authorized faculty members or administrators can access the content management dashboard to curate posts, while the public interface remains open for student consumption. This transition from ephemeral communication to a persistent web platform significantly enhances departmental transparency, improves information accessibility, and preserves "institutional memory" ensuring that valuable academic contributions and announcements remain available for future academic cohorts.

Keywords: Blogging Platform, Node.js, MongoDB, Full Stack Development, Departmental Communication, Web Security, Information Management System

LIST OF TABLES

Table	Title	Page
1.3.1	Hardware Specification	2
1.4.1	Software Specification	2

LIST OF FIGURES

Figure	Title	Page
3.2.1	Data Flow Diagram	9
3.2.2	Entity Relationship Diagram	10
3.3.3	Database Schema	11
4.1	User Login/ Sign Up Page	28
4.2	Feed Page	29
4.3	Create a Post Form	31
4.4	Profile Page	32
4.5	Group Page	32
4.6	Research Page	33
4.7	Search Page	33
4.8	API Test (Postman)	34
4.9	Developer Page	35
4.10	User Database	36
4.11	Profile Database	36
4.12	Research Database	37
4.13	Post Database	37
4.14	Group Database	38
4.15	Notification Database	38
4.16	Loading Page	39

LIST OF ABBREVIATIONS

Abbreviation	Full Form
SRS	Software Requirement Specification
WWW	World Wide Web
HTML	Hypertext Markup Language
CSS	Cascaded Style Sheet
SQL	Structured Query Language
API	Application Programming Interface
JWT	JSON Web Token
CMS	Content Management System
MERN	MongoDB, Express, React, Node.js

CHAPTER-1

INTRODUCTION

In contemporary university environments, seamless information flow between faculty, administration, and students is essential for fostering engagement, ensuring administrative coordination, and enhancing the overall learning experience. However, many departments still rely on a fragmented ecosystem of communication tools.

JLU Orbit is conceived as a dedicated digital ecosystem to resolve this challenge. It is designed to serve as the official, centralized platform for the Jagran School of Computer Applications (JSCA), empowering departments to publish structured content, maintain a permanent digital archive, and promote active student–faculty participation in a controlled, academic environment.

1.1 Problem Definition

The absence of a unified, department-specific communication system at JLU creates several operational inefficiencies:

- **Information Silos:** Critical announcements, academic resources, and event details become scattered across disparate channels (WhatsApp, email, notice boards).
- **Lack of Persistence:** Information shared in messaging apps or emails is difficult to retrieve later and is not formally archived, leading to a loss of institutional memory.
- **No Formal Mechanism:** There is no formal, branded platform for faculty to share academic articles or technical insights, diminishing the department's digital presence.

1.2 Project Overview

The primary objectives of the JLU Orbit project are as follows:

- **To Design and Develop** a secure, scalable, and responsive full-stack web application for departmental blogging.
- **To Implement** a robust authentication system to ensure that only authorized faculty or students can create, update, and manage content.
- **To Enable** content creators to publish rich-text posts, including embedded images and media, through an intuitive dashboard.
- **To Establish** a persistent, organized, and searchable digital archive of all published academic content and announcements.

The scope of this project is to deliver a Minimum Viable Product (MVP) that serves as the core communication platform. This includes user authentication, full CRUD (Create, Read, Update, Delete) functionality for posts, image uploads, and a public-facing front end to display all published posts.

1.3 Hardware Specification

Table 1.3.1: Hardware Specification (Development)

Component	Minimum Specification
Processor	Intel Core i5 (or equivalent AMD)
RAM	8 GB
Storage	256 GB SSD (for OS, IDE, and dependencies)
Network	Stable Broadband Internet Connection

1.4 Software Specification

Table 1.4.1: Software Specification (Development)

Category	Software/Technology	Purpose
Backend Runtime	Node.js (v22.21.1)	Server-side JavaScript execution
Backend Framework	Express.js	API routing and middleware management
Database	MongoDB (Atlas)	NoSQL database for content storage
Frontend	HTML5, CSS3, JavaScript (EJS)	User interface and presentation layer
API Testing	Postman	Endpoint testing and validation
IDE	Visual Studio Code	Code editing and debugging
Version Control	Git & GitHub	Source code management

CHAPTER-2

RELATED WORK

2.1 Existing System

The current communication and information-sharing framework within the FAST (Faculty of Applied Sciences & Technology) department at JLU is fragmented, unstructured, and highly dependent on third-party consumer tools. These platforms were never designed for institutional or academic communication, resulting in inefficiencies, data loss, and lack of accountability. The existing methods can be summarized as follows:

1. WhatsApp Groups

These are widely used among students and occasionally by faculty for immediate or informal announcements.

Limitations:

- Zero archival capability: Important information quickly disappears in continuous chat streams.
- Poor searchability: Retrieving old messages or files is inefficient and often impossible.
- Lack of categorization: Academic notices, doubts, memes, and unrelated discussions mix in the same feed.
- Platform dependency: Users must share their personal mobile numbers, which compromises privacy.
- Not suitable for formal or official communication.

2. Faculty Emails

Faculty distribute official circulars, assignment sheets, and academic notices via email.

Limitations:

- Information silos: Emails stay inside individual inboxes, meaning the department has no centralized archive.
- Zero community visibility: Students cannot see updates unless explicitly mailed.
- Difficult to track: Important emails get buried below personal or promotional messages.
- No interaction: Students cannot like, comment, or discuss academic posts.

3. Physical Notice Boards

Physical boards remain the official channel for university-level announcements and examination notices.

Limitations:

- Requires physical presence in the department building.
- No digital backup: Notices cannot be searched, saved, or shared automatically.
- Static and outdated: Once removed, the information is permanently lost.
- Slow dissemination for urgent updates.

Summary of Gap

The current system suffers from:

- Lack of centralization
- No searchable digital archive
- Poor dissemination of academic events
- No collaboration or community engagement
- Inability to support research sharing, project groups, or professional discussions

These limitations highlight the need for an all-in-one, academically structured platform like JLU Orbit, built specifically for the FAST department.

2.2 Proposed System (JLU Orbit)

JLU Orbit addresses the shortcomings of the existing communication methods by introducing a dedicated digital ecosystem, designed from the ground up to support an academic community. It functions as a three-tier, cloud-enabled web platform that integrates authentication, posts, groups, research sharing, and real-time notifications.

Core Components

1. Secure Authentication

- Login and signup system using bcrypt for password hashing.
- OTP-based email verification using Brevo ensures only legitimate JLU users can join.
- JWT-based authentication secures all protected routes and ensures session integrity.

2. Centralized Content Dashboard

- Students and faculty access a private dashboard where they can create, edit, and manage posts.
- Post categories include discussions, doubts, announcements, research, events, and project work.
- Users have personal profiles linked with their academic identity (course, specialization, semester).

3. Dynamic Post System

- The system fetches posts from MongoDB and renders them dynamically on the frontend.
- Features include:
 - Likes and comments
 - Share counters
 - Tag-based classification
 - Real-time updates via Socket.io
 - Image uploads using Cloudinary + multer

4. Group Collaboration Module

- Users can create both public and private groups for:
 - Project teams
 - Hackathons
 - Research collaborations
 - Study groups
 - Academic clubs
- Each group maintains its own feed and member management.

5. Research Hub

- Users can upload research papers, abstracts, and attachments.
- Keywords and metadata enable better academic discovery.

6. Real-Time Notification System

- Users receive instant notifications on:
 - Likes and comments
- Delivered through Socket.io for live updates.

7. Full Data Archival

- Every post, comment, and notice stays stored in MongoDB, enabling:
 - Long-term departmental archive
 - Easy searchability
 - Reference for future batches

Impact of Proposed System

JLU Orbit eliminates fragmentation by consolidating all departmental communication into a single, structured, academically relevant platform.

It bridges the gap between formal communication and student engagement, unlike WhatsApp or email-only systems.

2.3 Feasibility Study

2.3.1 Technical Feasibility

The project is technically feasible due to its robust and modern technology stack:

- Node.js + Express provide an event-driven, scalable backend.
- MongoDB supports flexible document storage suitable for posts, comments, groups, and notifications.
- Socket.io allows real-time communication without complex infrastructure.
- Cloudinary handles media uploads efficiently.
- Brevo API ensures reliable OTP delivery.

All technologies used are:

- Open-source
- Widely adopted

- Compatible with each other
- Highly scalable for future expansion (more users, more features)

No specialized hardware or paid software is required.

2.3.2 Operational Feasibility

The system is operationally viable because:

- The user interface is simple, intuitive, and optimized for students and faculty.
- Core operations (posting, liking, joining groups) require no technical expertise.
- Content moderators and faculty can manage announcements without training.
- Maintenance is simplified due to:
 - Modular architecture
 - Consistent JavaScript ecosystem (Node.js + browser JS)
 - Clear separation between frontend, backend, and database

Daily operation of the system requires minimal administrative overhead.

2.3.3 Economic Feasibility

The system is highly cost-effective:

- All technologies used (Node.js, Express, MongoDB, Socket.io) are free and open-source.
- The university incurs zero licensing fees.
- Deployment options:
 - University's own Linux server
 - Low-cost cloud hosting (e.g., Railway, Render, DigitalOcean)
- Media storage is pay-per-use on Cloudinary, but student images are low-cost due to small size.

Because development and maintenance require only standard web development knowledge, long-term operational cost remains near zero.

CHAPTER-3

SYSTEM ANALYSIS & DESIGN

3.1 Requirement Specification

3.1.1 Functional Requirements

- **F1: User Authentication**
 - The system shall provide a secure login page for students/faculty.
 - The system shall validate user credentials against the database.
 - The system shall use password hashing (e.g., bcrypt).
 - The system shall generate a JWT token upon successful login.
- **F2: Post Management (CRUD)**
 - Authenticated users shall be able to create new posts (Title, Content, Image).
 - Authenticated users shall be able to view a list of all posts they have created.
 - Authenticated users shall be able to update their existing posts.
 - Authenticated users shall be able to delete their existing posts.
- **F3: Public Interface**
 - The system shall display a list of all published posts to any visitor.
 - Posts shall be displayed in reverse chronological order.

3.1.2 Non-Functional Requirements

- **NFR1: Security:** All routes for creating or modifying data must be protected and require a valid JWT.
- **NFR2: Performance:** The public-facing blog page must load in under 3 seconds.
- **NFR3: Usability:** The student profile page must be clean, intuitive, and easy to navigate.
- **NFR4: Maintainability:** The backend code must be structured in a modular (e.g., MVC - Model-View-Controller) pattern.
- **NFR5: Reliability:** The system shall handle common errors (e.g., database connection failure, invalid input) gracefully.

3.2 Flowchart

3.2.1. Data Flow Diagram

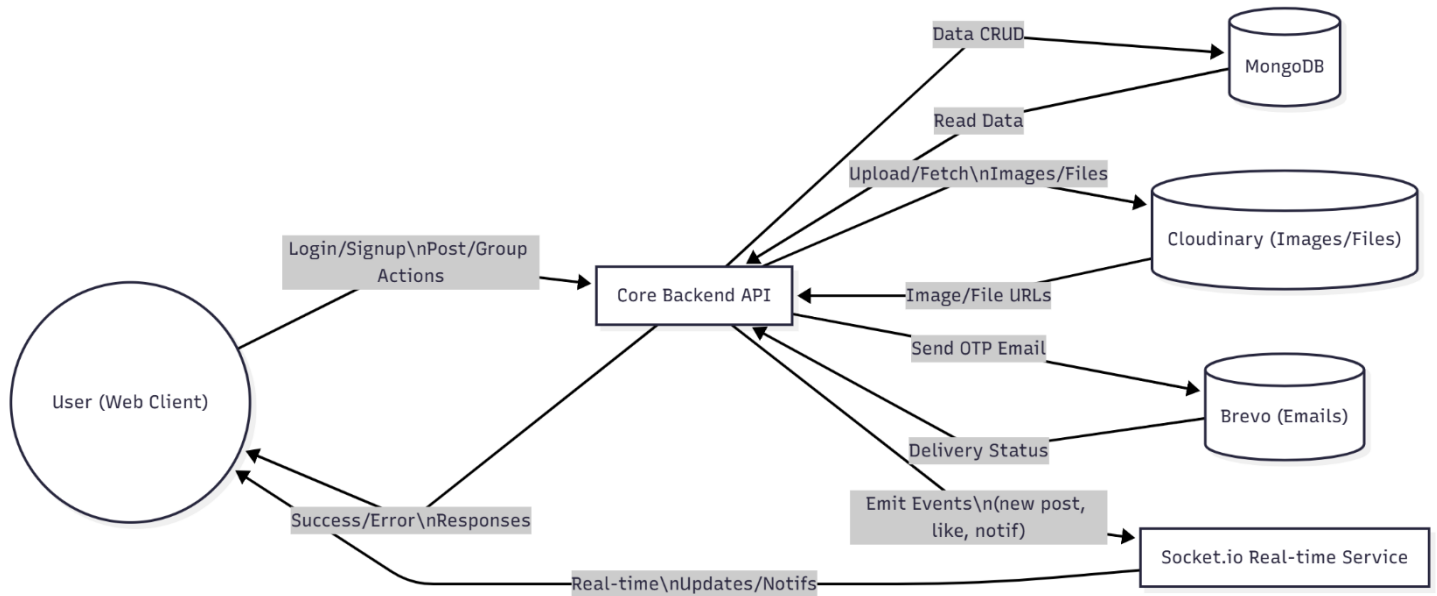


Figure: 3.2.1 Data Flow Diagram (DFD)

Description: This architecture diagram illustrates the workflow of a web application where a **User (Web Client)** interacts with a central **Core Backend API** to perform actions like logging in or posting content. The backend orchestrates data management by performing CRUD operations on a **MongoDB** database and handling media file storage and retrieval via **Cloudinary**. For external communication, the system uses **Brevo** to deliver transactional emails (such as OTPs) and checks their delivery status. Additionally, a **Socket.io** service is integrated to manage real-time events, pushing updates and notifications back to the user client to complete the interactive loop.

3.2.2 Entity-Relationship Diagram

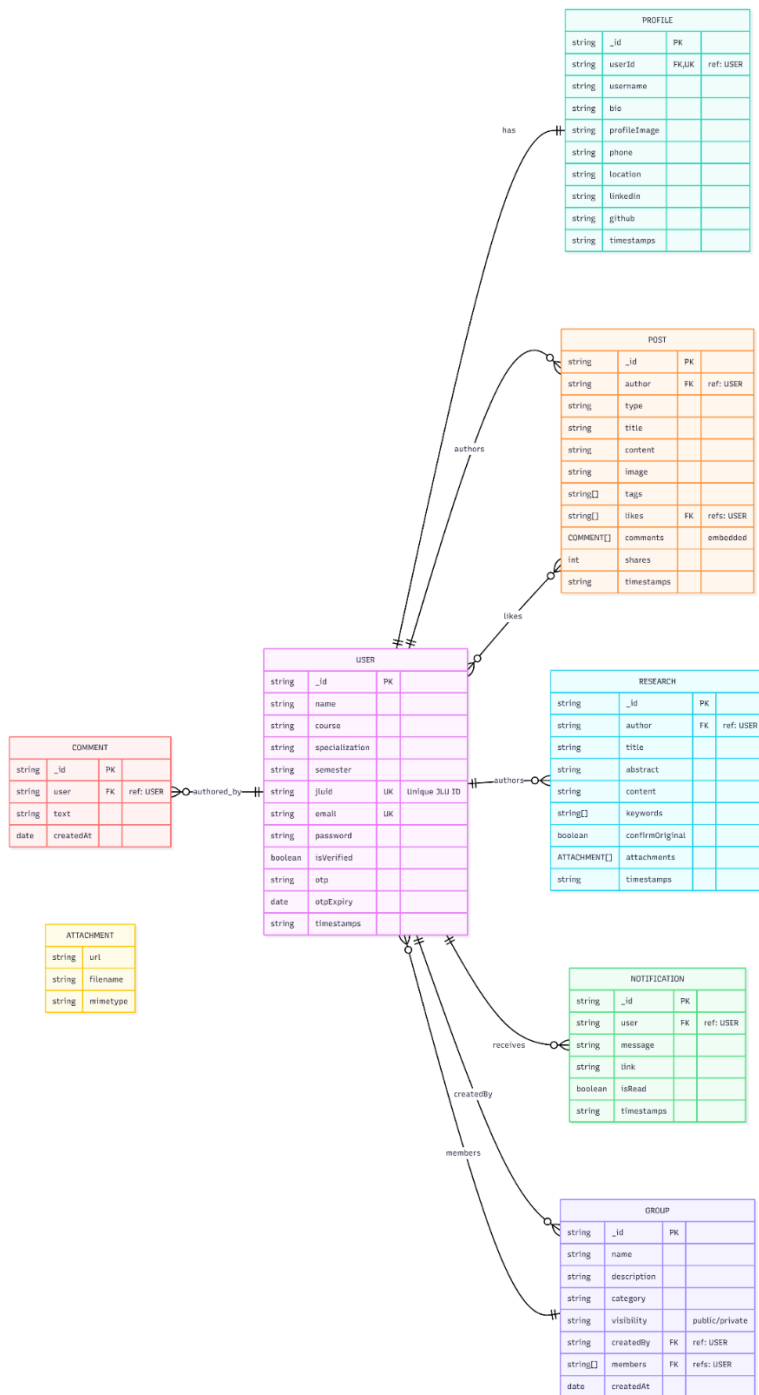


Figure: 3.2.2 Entity-Relationship (ER) Diagram

Description: This Entity Relationship Diagram (ERD) outlines the database schema for a social or collaborative platform, centered on a User entity that connects to Profiles, Posts, Research, and Groups. It visualizes the relationships between these data models—showing, for instance, how a single user can author multiple posts or join various groups—while defining specific attributes like data types (strings, booleans) and primary keys. The structure supports key application features such as authentication, content creation (with embedded comments and attachments), and real-time notifications.

3.2.3 Database Schema (MongoDB)

The system utilizes two primary collections in MongoDB:

1. users Collection

```
1  const mongoose = require("mongoose");
2
3  const UserSchema = new mongoose.Schema(
4    {
5      name: { type: String, required: true },
6      course: { type: String, required: true },
7      specialization: { type: String, required: true },
8      semester: { type: String, required: true },
9      jfluid: { type: String, required: true, unique: true },
10     email: { type: String, required: true, unique: true },
11     password: { type: String, required: true },
12
13     isVerified: { type: Boolean, default: false },
14     otp: { type: String },
15     otpExpiry: { type: Date },
16   },
17   { timestamps: true }
18 );
19
20 module.exports = mongoose.model("User", UserSchema);
21
```

Figure: 3.2.3 User Collection Schema

2. posts Collection

```
const mongoose = require("mongoose");

const CommentSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  text: { type: String, required: true },
  createdAt: { type: Date, default: Date.now }
});

const PostSchema = new mongoose.Schema(
  {
    author: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
    type: { type: String, required: true },
    title: { type: String, required: true },
    content: { type: String, required: true },
    image: { type: String },
    tags: [{ type: String }],

    likes: [{ type: mongoose.Schema.Types.ObjectId, ref: "User" }],
    comments: [CommentSchema],

    shares: {
      type: Number,
      default: 0
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model("Post", PostSchema);
```

Figure: 3.2.3 Post Collection Schema

3.3 Design and Test Steps / Criteria

1. System Design Steps

1.1 Requirement Analysis

- Identify all user roles and actions: registration, login, posting, liking, commenting, joining groups, uploading research, receiving notifications.
- Define non-functional requirements: security, responsiveness, scalability, and real-time updates.
- Map out external service dependencies:
 - Cloudinary for media uploads
 - Brevo for email OTP
 - Socket.io for real-time functionality

1.2 Database Schema Design

- Define all collections and fields based on platform needs:
 - User, Profile, Post, Group, Research, Notification
- Establish relationships:
 - 1:1 (User ↔ Profile)
 - 1:many (User → Posts, User → Research, User → Notifications)
 - many:many (Posts ↔ Users through likes, Group ↔ Members)
- Use embedded subdocuments for lightweight structures (e.g., Post comments).
- Create indexes for email, juid, post tags, and searchable fields.

1.3 API Architecture Design

- RESTful endpoints for all resources:
 - /auth, /posts, /groups, /research, /notifications
- Each endpoint designed with:
 - Input validation
 - Authentication middleware

- Error handling workflow
- JSON response structure
- Define allowed methods: GET, POST, PUT, DELETE
- Outline access control rules (auth-only routes, role-based actions)

1.4 Frontend Architecture Design

- Design pages and UI components:
 - Authentication page
 - Feed page
 - Groups dashboard
 - Research upload page
 - Notification drawer
 - User profile page
- Split logic into modular JS files under /html/js/.
- Implement client-side caching of JWT tokens.

1.5 Real-Time System Design

- Integrate Socket.io for:
 - New post broadcast
 - New like/comment events
 - Group activity notifications
- Maintain stateful user connections mapped to user IDs.

1.6 Security Design

- Hash user passwords using bcrypt.
- Use JWT for session authentication.
- Secure Cloudinary uploads (size/type validation).
- Prevent unauthorized access with role-based guards.

- Avoid storing OTP after verification.

2. Component-Level Design Steps

2.1 Authentication Module

- Register flow:
 - Validate input → Create user → Send OTP → Verify email → Create profile.
- Login flow:
 - Check credentials → Issue JWT → Return session credentials.

2.2 Post Management Module

- Create Post:
 - Verify token → Upload media → Save post → Notify via Socket.io.
- Like Post:
 - Add/remove userId from likes array → Create notification.
- Comment Post:
 - Append embedded comment → Notify author.

2.3 Group Module

- Create group:
 - Verify creator → Save group → Add creator to members.
- Join group:
 - Push new member ID into group.members.

2.4 Research Module

- Upload research:
 - Verify author → Validate files → Upload attachments → Save research entry.

2.5 Notification Module

- Create notification record for each action:

- Likes, comments, group joins, new posts.
- Deliver notification via:
 - Database storage
 - Real-time socket event

3. Test Steps / Test Criteria

3.1 Authentication Testing Criteria

Functional Tests

- Registration succeeds with valid input.
- Duplicate email or JLU ID returns proper error.
- OTP is generated, sent, and expires correctly.
- Login rejects invalid credentials.
- Login returns valid JWT on success.

Security Tests

- Password is stored only after hashing (never raw).
- JWT is validated for all protected routes.
- Users cannot access other users' profiles without permission.

Edge Case Tests

- Resend OTP works only for unverified accounts.
- OTP expires after set duration.
- Empty or malformed inputs return error 400.

3.2 Post Module Testing Criteria

Functional Tests

- Post creation works with/without images.

- Tags are saved and searchable.
- Likes update real count and prevent duplicates.
- Comments save under the correct post.

Real-Time Tests

- New posts broadcast to all connected users.
- Likes and comments update instantly via sockets.

Data Integrity Tests

- Embedded comments maintain correct timestamp and user reference.
- Invalid image types are blocked as expected.

3.3 Group Module Testing Criteria

Functional Tests

- Group creation validates required fields.
- Only creator is auto-added as member.
- Members can successfully join groups.
- Private groups enforce visibility restrictions.

Permission Tests

- Non-admin cannot modify or delete group info.
- Unauthorized user cannot join private groups without approval.

3.4 Research Module Testing Criteria

Functional Tests

- Research upload supports multiple attachments.
- Metadata (title, abstract, keywords) saves correctly.

- Originality confirmation works as Boolean.

File Handling Tests

- Invalid file formats rejected.
- Attachment metadata stored correctly.

3.5 Notification Testing Criteria

Functional Tests

- Notifications created for all major actions:
 - Likes, comments, group creation, joins.
- Notifications marked as read/unread correctly.
- Clicking notification leads to correct page (via link).

Real-Time Tests

- User receives notification instantly via socket.

3.6 API Testing Criteria

General Tests

- All endpoints return consistent JSON structure.
- All errors include message + status code.
- Rate of response under acceptable limits (typically < 200ms).

Validation Tests

- Payload validation using custom logic or middleware.
- Missing or invalid fields return error 400/422.

3.7 Database Testing Criteria

- Unique constraints enforced for email and jluuid.

- Referential integrity maintained for all ObjectId references.
- Indexes functional for search fields.
- Data insertion on load test remains under 16MB document size.

3.8 UI/UX Testing Criteria

- All buttons, forms, and modals responsive.
- Feed updates instantly without page refresh.
- Profile and group pages load all dynamic data correctly.

3.9 Cross-System Testing

Cloudinary Integration

- Ensure successful upload and correct URL storage.
- Validate image MIME types.

Brevo Integration

- OTP emails received consistently.
- Retry logic works for failed email sends.

Socket.io Integration

- Stable WebSocket connection.
- Automatic reconnection handling.

4. Acceptance Criteria Checklist

- Registration, OTP verification, and login work end-to-end.
- Posts can be created, liked, commented, and displayed in real-time.
- Groups support creation and membership logic.
- Research uploads with attachments store correctly.
- Notifications deliver instantly and persist in database.
- All MongoDB schemas follow documented rules.
- Frontend UI is responsive and consistent.
- No unauthorized access possible.
- System stable under multiple concurrent users.

3.4 Algorithms and Pseudo Code

3.4.1 User Registration and OTP Verification

Algorithm

1. Receive user details.
2. Validate inputs and check duplicates.
3. Hash password and create user with isVerified = false.
4. Generate OTP and expiry time.
5. Email OTP to user.
6. On verification, match OTP and activate user account.

Register(data):

IF invalid OR exists: error

hash = Hash(data.password)

user = Create(data + hash + isVerified=false)

otp = GenerateOTP()

SaveOTP(user, otp)

SendEmail(user.email, otp)

Verify(email, otp):

user = Find(email)

IF mismatch OR expired: error

user.isVerified = true

ClearOTP(user)

3.4.2 Login (JWT)

Algorithm

1. Validate user credentials.

2. Compare hashed password.
3. Generate JWT on success.

Login(identifier, password):

user = Find(identifier)

IF invalid: error

IF !Match(password, user.password): error

token = GenerateJWT(user.id)

3.4.3 Create Post (with Image Upload)

Algorithm

1. Authenticate user.
2. Validate post fields.
3. Upload image (if present).
4. Save post to database.
5. Emit real-time new_post.

CreatePost(user, data, file):

IF file: url = Upload(file)

post = NewPost(user, data, url)

Emit("new_post", post)

3.4.4 Like / Unlike Post

Algorithm

1. Check if user already liked the post.
2. Add/Remove like accordingly.
3. Notify post author on like.

4. Emit like update.

ToggleLike(post, user):

IF user in post.likes: Remove

ELSE: Add & NotifyAuthor()

Emit("like_update")

3.4.5 Comment on Post

Algorithm

1. Validate comment.
2. Append to post's comment list.
3. Notify post author.
4. Emit real-time update.

CreateGroup(user, data):

group = NewGroup(data + createdBy=user)

group.members.add(user)

3.4.6 Create Group

Algorithm

1. Validate group details.
2. Save group with creator as member.

CreateGroup(user, data):

group = NewGroup(data + createdBy=user)

group.members.add(user)

3.4.7 Join Group

Algorithm

1. Check membership.

2. If public → add user.
3. If private → send join request.

JoinGroup(group, user):

IF user in members: return

IF group.public: AddMember(user)

ELSE: CreateJoinRequest()

3.4.8 Upload Research

Algorithm

1. Validate metadata.
2. Upload attachments.
3. Save research document.

UploadResearch(user, data, files):

urls = UploadAll(files)

SaveResearch(user, data, urls)

3.4.9 Notification Creation

Algorithm

1. Triggered on likes, comments, groups, or posts.
2. Create notification record.
3. Emit real-time notification.

Notify(user, msg, link):

CreateNotification(user, msg, link)

EmitToUser(user, "notification")

3.4.10 Real-Time Socket Updates

Algorithm

1. Authenticate socket connection.
2. Join user room.
3. Emit events for posts, likes, comments, and notifications.

OnSocketConnect(token):

```
user = Verify(token)
```

```
JoinRoom("user_" + user.id)
```

3.4.11 Feed Retrieval

Algorithm

1. Apply filters (tags/type).
2. Sort by newest.
3. Return paginated posts.

GetFeed(filters):

```
posts = Query(filters).SortByDate().Paginate()
```

3.4.12 Search

Algorithm

1. Search posts (text search), users (name), and groups.
2. Return matched results.

SearchAll(query):

```
posts = SearchPosts(query)
```

```
users = SearchUsers(query)
```

```
groups = SearchGroups(query)
```

3.4.13 Profile Update

Algorithm

1. Validate update fields.
2. Upload image if provided.
3. Update profile document.

UpdateProfile(user, data, file):

IF file: url = Upload(file)

Update(user.profile, data + url)

3.5 Testing Process

A multi-level testing strategy was employed to ensure application quality.

1. Unit Testing

Unit testing focused on individual backend API endpoints using **Postman**.

Examples:

- **Login API Test:**
Input: Wrong email/password
Expected: 401 Unauthorized with error message
- **OTP Verification Test:**
Input: Expired or incorrect OTP
Expected: "OTP expired/invalid"

Modules Unit Tested:

- Authentication (Register, OTP, Login)
- Post creation, like, comment
- Group create/join APIs
- Research upload validator
- Notification creator

2. Integration Testing

Integration tests validated the flow between **Frontend** → **Backend** → **Database** → **Cloudinary/Brevo** → **Socket.io**.

Examples:

- **Create Post Flow:**
 - User submits form
 - Image uploaded to Cloudinary
 - Post stored in MongoDB
 - Real-time update appears on dashboard

- **Registration + OTP Flow:**

- User registers → OTP email sent → User verifies → Profile auto-created

- **Group Join Flow:**

- User joins group → Added to members[] → Notification sent → UI updates

3. System Testing

System testing ensured all modules worked together.

Key Areas Checked:

- Dashboard feed loading
- Search (posts/users/groups)
- Research upload with multiple attachments
- Notifications and real-time events
- JWT token handling and access control

All major features were verified end-to-end.

4. User Acceptance Testing (UAT)

The system was reviewed by:

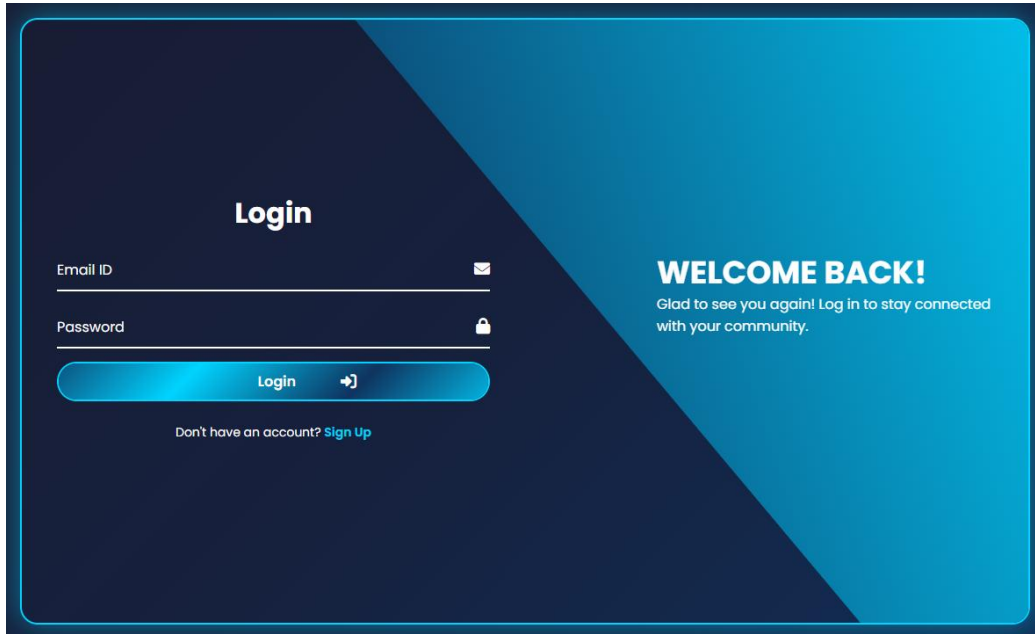
- Project supervisor
- Selected students

Outcome:

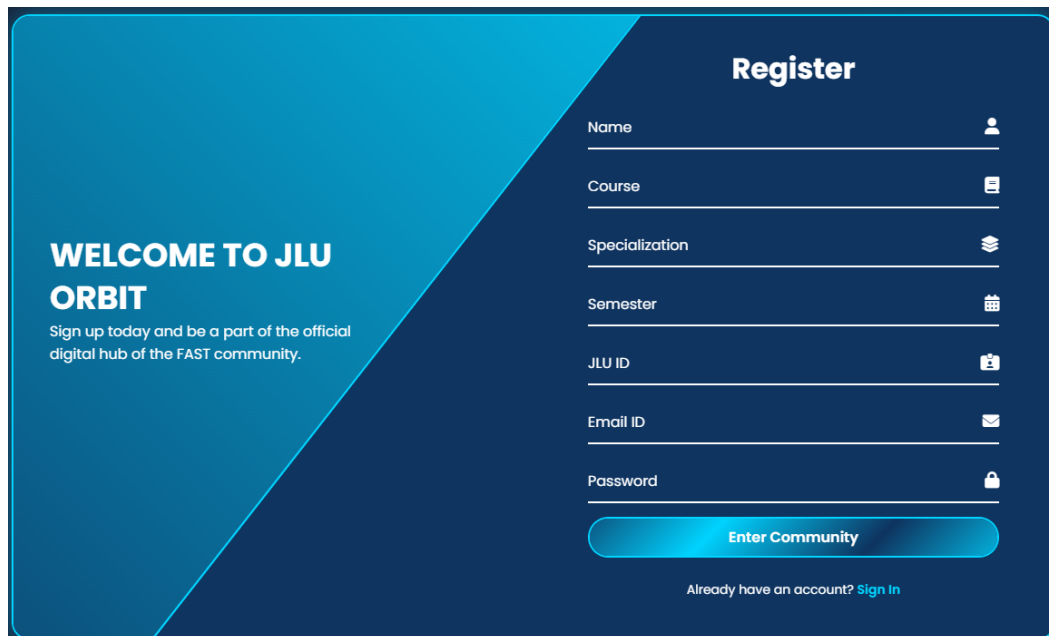
- Features matched requirements
- UI found easy to use
- Real-time updates worked smoothly
- Minor UI fixes implemented.

CHAPTER-4

RESULTS / OUTPUTS



The login page features a dark blue background with a light blue diagonal stripe. On the left, the 'Login' title is centered above two input fields: 'Email ID' with an envelope icon and 'Password' with a lock icon. Below these is a blue 'Login' button with a right arrow icon. A link 'Don't have an account? Sign Up' is positioned below the button. On the right, the text 'WELCOME BACK!' is displayed in bold, followed by a message: 'Glad to see you again! Log in to stay connected with your community.'



The register page has a dark blue background with a light blue diagonal stripe. On the left, the text 'WELCOME TO JLU ORBIT' is shown in bold, with a message below: 'Sign up today and be a part of the official digital hub of the FAST community.' On the right, the 'Register' title is centered above seven input fields: 'Name' (person icon), 'Course' (book icon), 'Specialization' (stack of books icon), 'Semester' (calendar icon), 'JLU ID' (ID card icon), 'Email ID' (envelope icon), and 'Password' (lock icon). A blue 'Enter Community' button is located below the fields. At the bottom, a link 'Already have an account? Sign In' is provided.

Figure 4.1: User Login/Sign Up Page

Description: The main entry point for students and faculties. It accepts an email and password to authenticate the user before granting access to the dashboard.

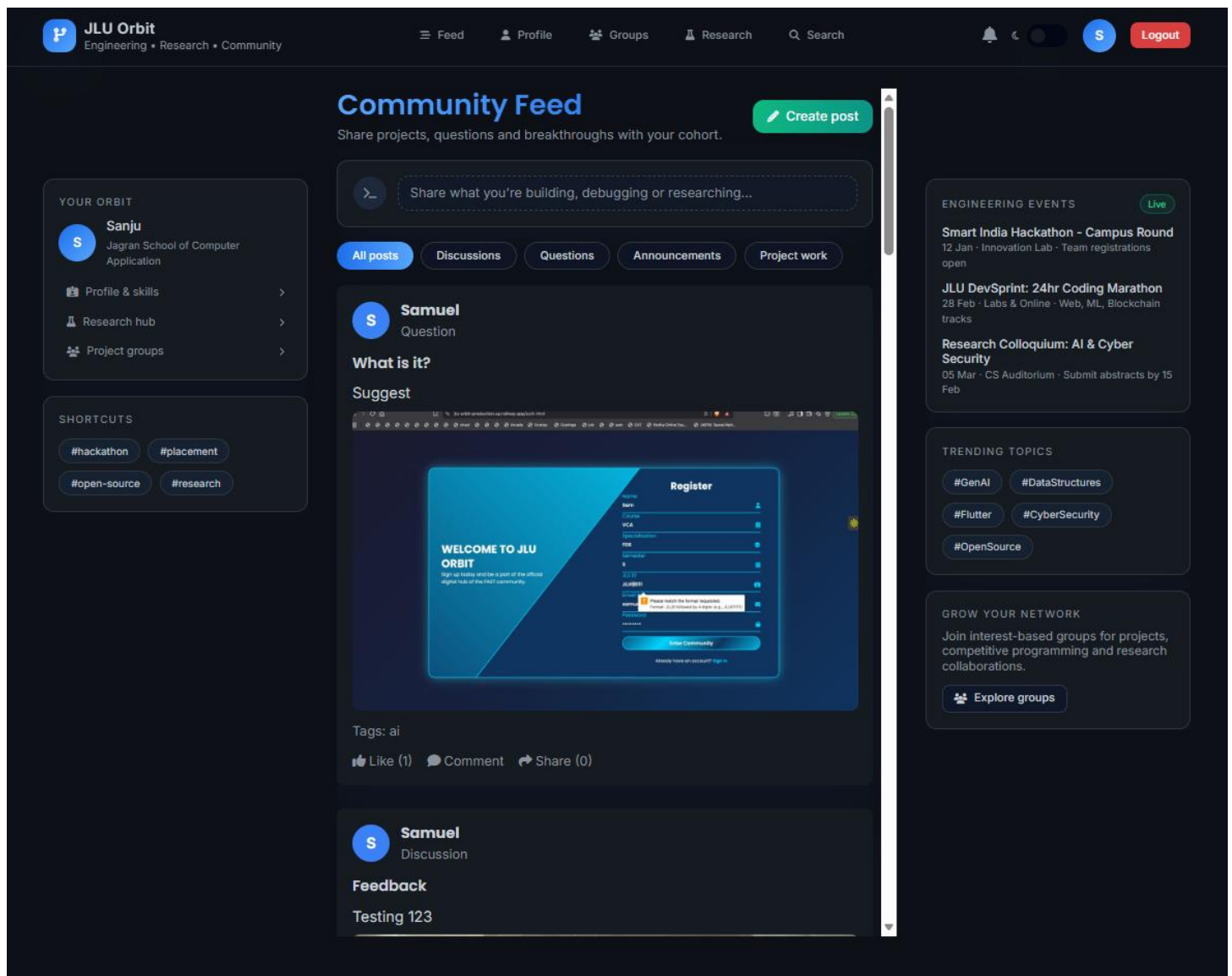


Figure 4.2.1: Feed Page (Dark Mode)

Description: The final output of the project, dynamically rendering all content from the database for public consumption. Posts are displayed in reverse chronological order.

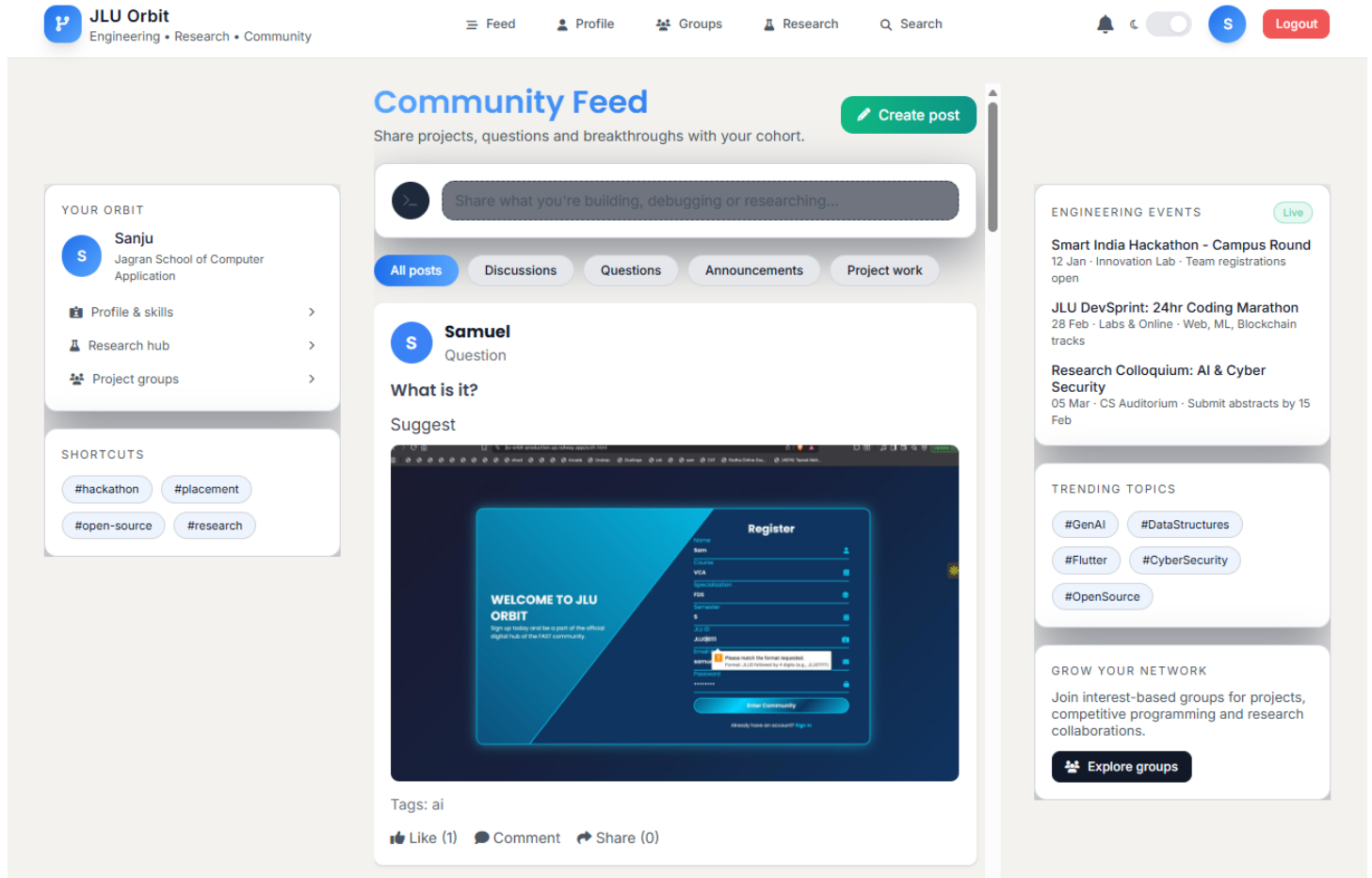
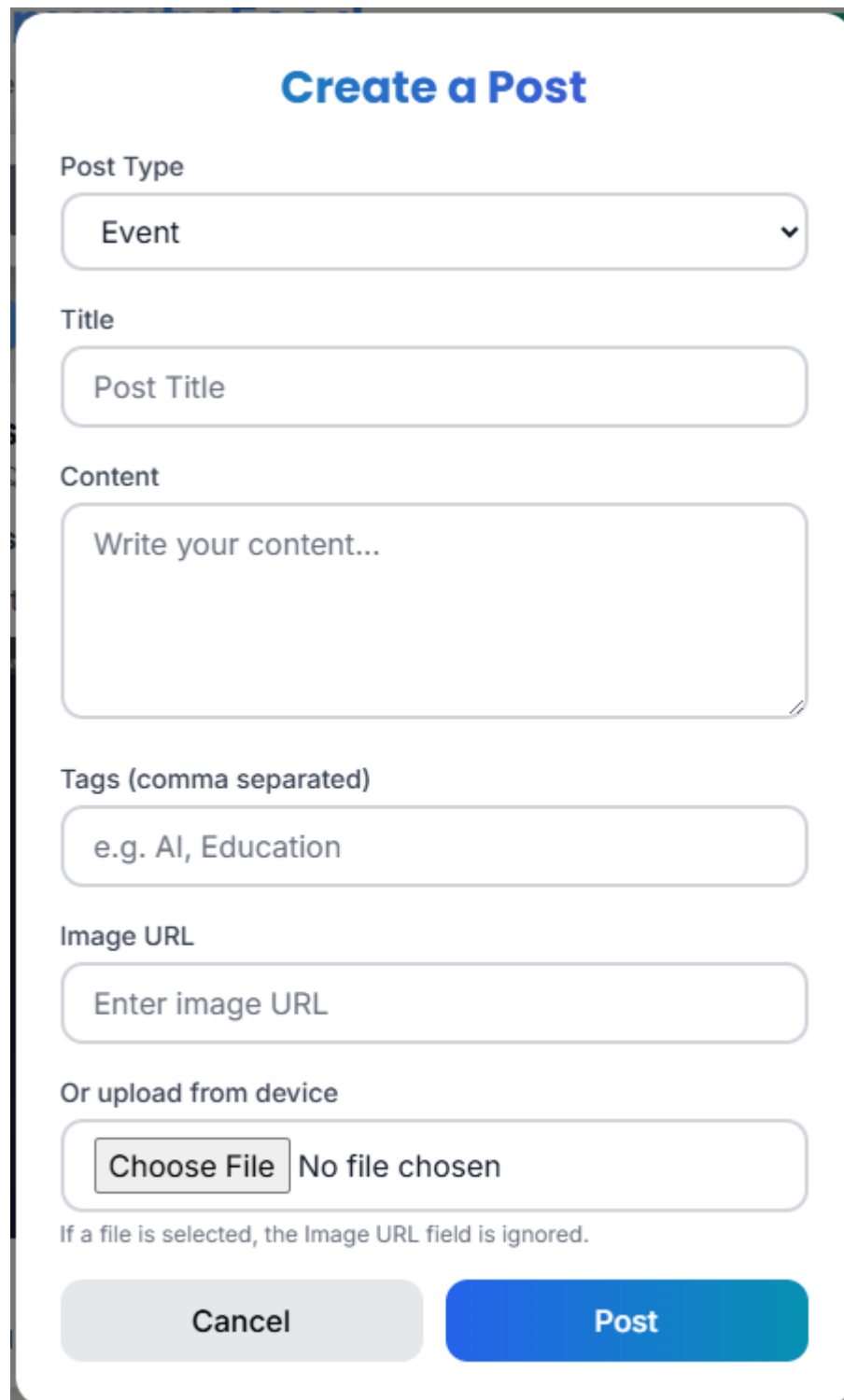


Figure 4.2.2: Feed Page (Light Mode)

Description: The final output of the project, dynamically rendering all content from the database for public consumption. Posts are displayed in reverse chronological order.



The image shows a mobile app interface for creating a post. At the top, the title "Create a Post" is displayed in a bold blue font. Below the title, the form is organized into several sections. The "Post Type" section features a dropdown menu with "Event" selected. The "Title" section has a text input field with the placeholder "Post Title". The "Content" section is a large text area with the placeholder "Write your content...". The "Tags (comma separated)" section has a text input field with the placeholder "e.g. AI, Education". The "Image URL" section has a text input field with the placeholder "Enter image URL". Below this, the "Or upload from device" section contains a file selection interface with a "Choose File" button and the text "No file chosen". A small note states, "If a file is selected, the Image URL field is ignored." At the bottom, there are two buttons: a grey "Cancel" button and a blue "Post" button.

Create a Post

Post Type

Event

Title

Post Title

Content

Write your content...

Tags (comma separated)

e.g. AI, Education

Image URL

Enter image URL

Or upload from device

Choose File No file chosen

If a file is selected, the Image URL field is ignored.

Cancel Post

Figure 4.3: Create New Post Form

Description: The interface for content creation. It uses a multipart/form-data form to allow text and image uploads simultaneously.

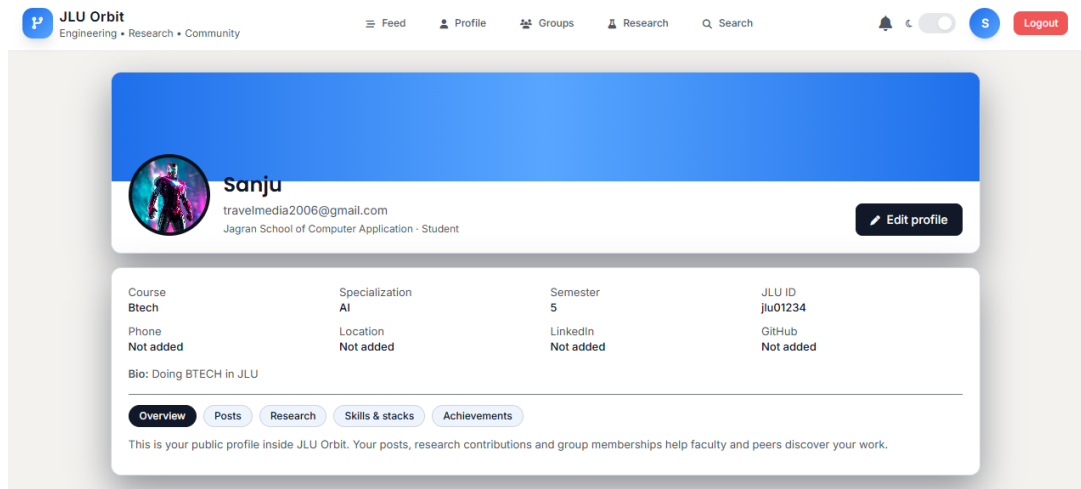


Figure 4.4: Profile Page

Description: Validating the backend API for post creation, independent of the frontend. This test confirms the server logic is working correctly.

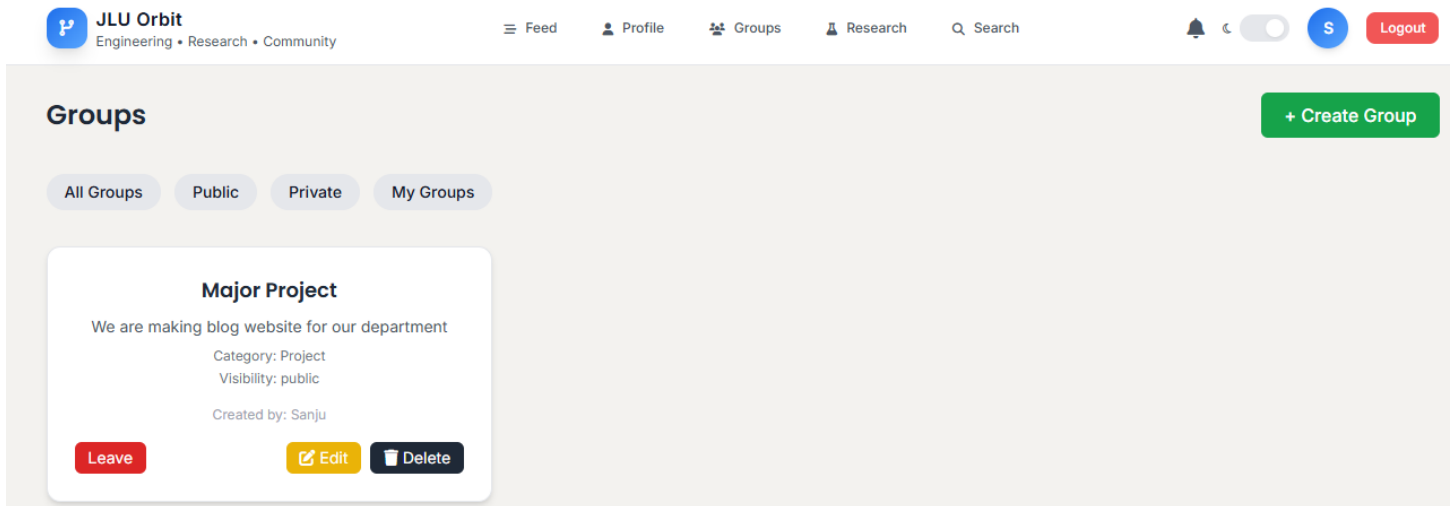


Figure 4.5: Group Page

Description: This image captures the "Groups" interface of the JLU Orbit web platform, designed for managing engineering and research communities with features to create, filter, and view groups. It displays a specific "Major Project" group card that includes details such as the project category and visibility, along with options for the user to edit, delete, or leave the group.

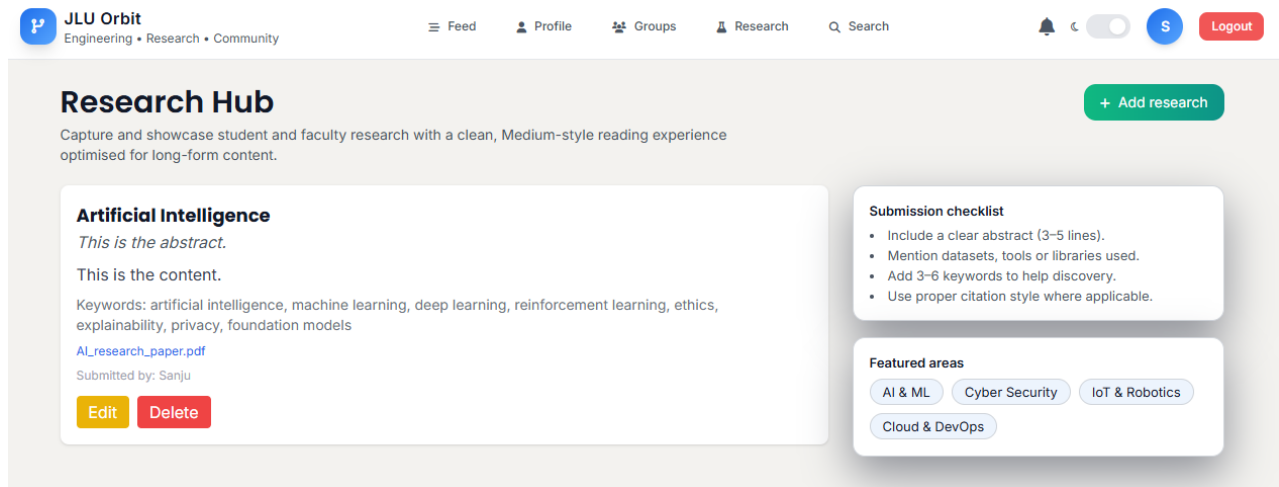


Figure 4.6: Research Page

Description: This image displays the "Research Hub" page of the JLU Orbit platform, a repository for student and faculty research that allows users to view, manage, and submit academic papers. The interface features a sample entry on Artificial Intelligence alongside a submission checklist and featured topic tags for navigation.

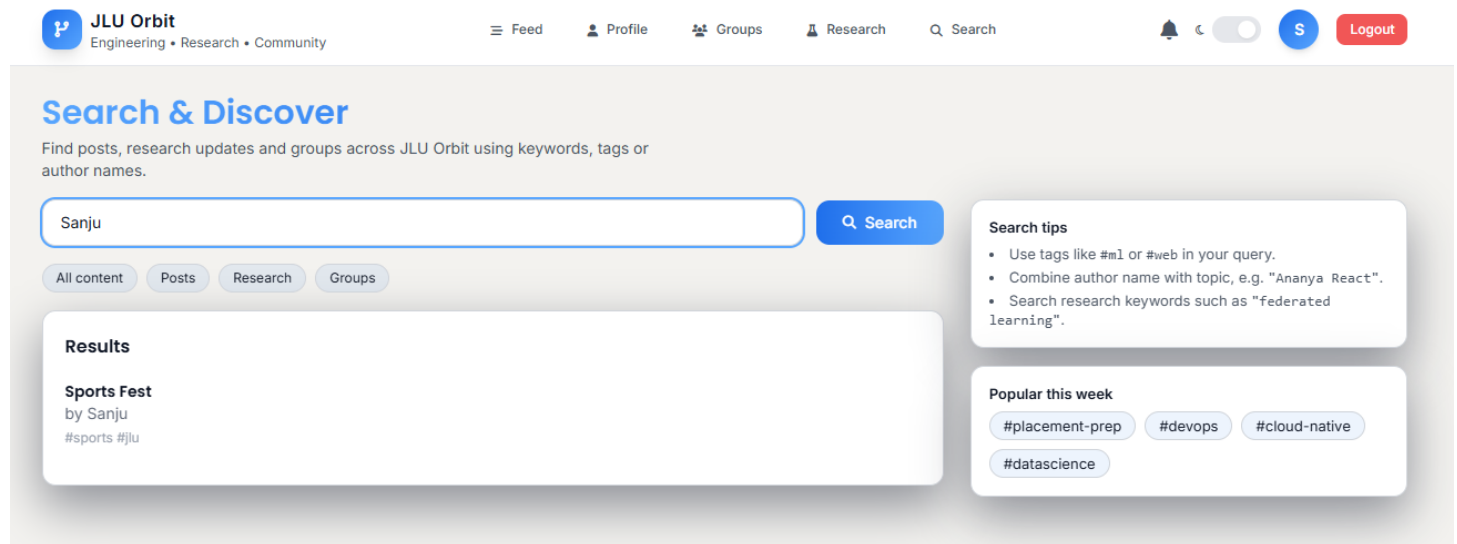


Figure 4.7: Search Page

Description: This image shows the "Search & Discover" interface of the JLU Orbit platform, designed for finding content across posts, research, and groups using keywords, tags, or author names. It features a search bar with active filters, a results feed displaying a specific query (e.g., "Sanju"), and sidebar widgets for search tips and trending topics.

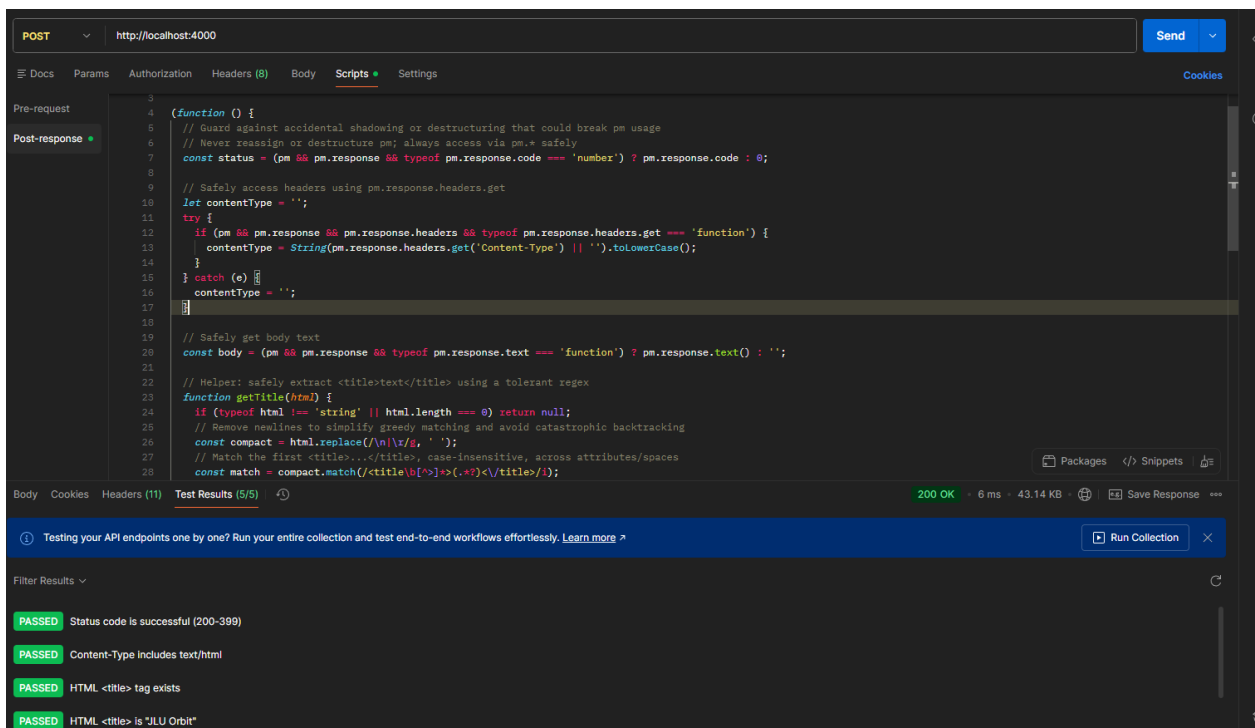


Figure 4.8: API Test (Postman)

Description: Validating the backend API for post creation, independent of the frontend. This test confirms the server logic is working correctly.

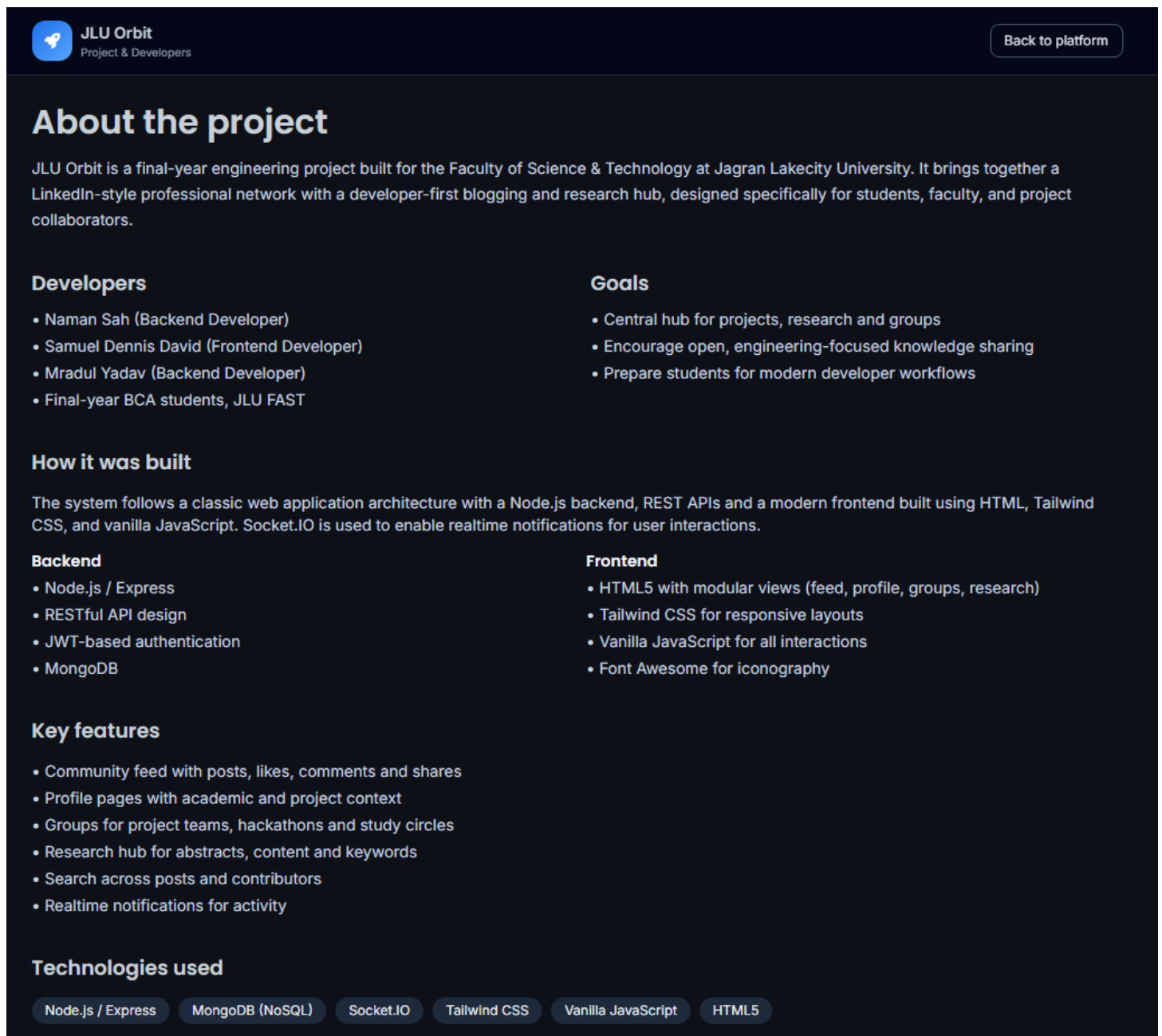


Figure 4.9: Developer Page

Description: This image presents the "About the project" page for JLU Orbit, a final-year engineering platform designed for Jagran Lakecity University to facilitate professional networking and research sharing. It lists the development team, project goals, and technical details, including the "How it was built" section and the specific technology stack like Node.js and MongoDB.

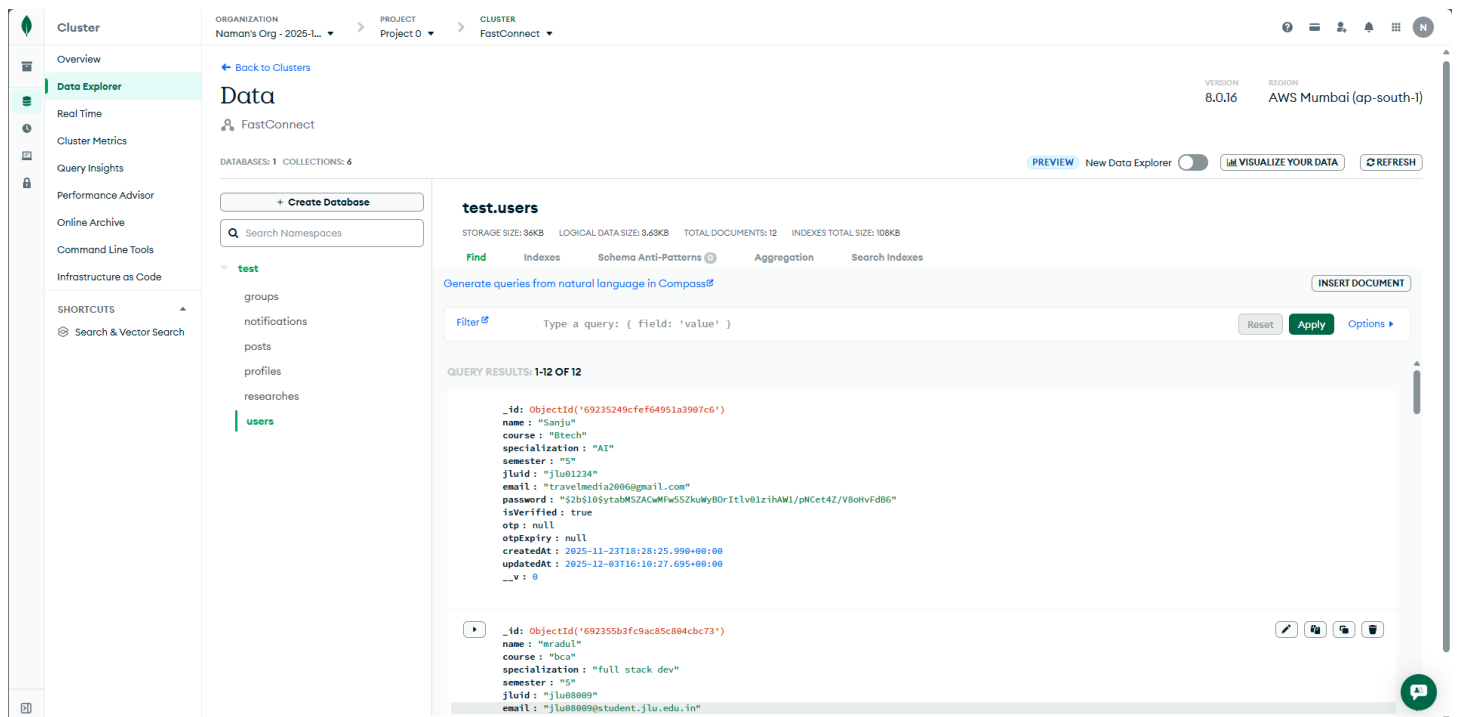


Figure 4.10: Users Database

Description: In the images stored data of users are shown in the JSON format which consist key:value pairs.

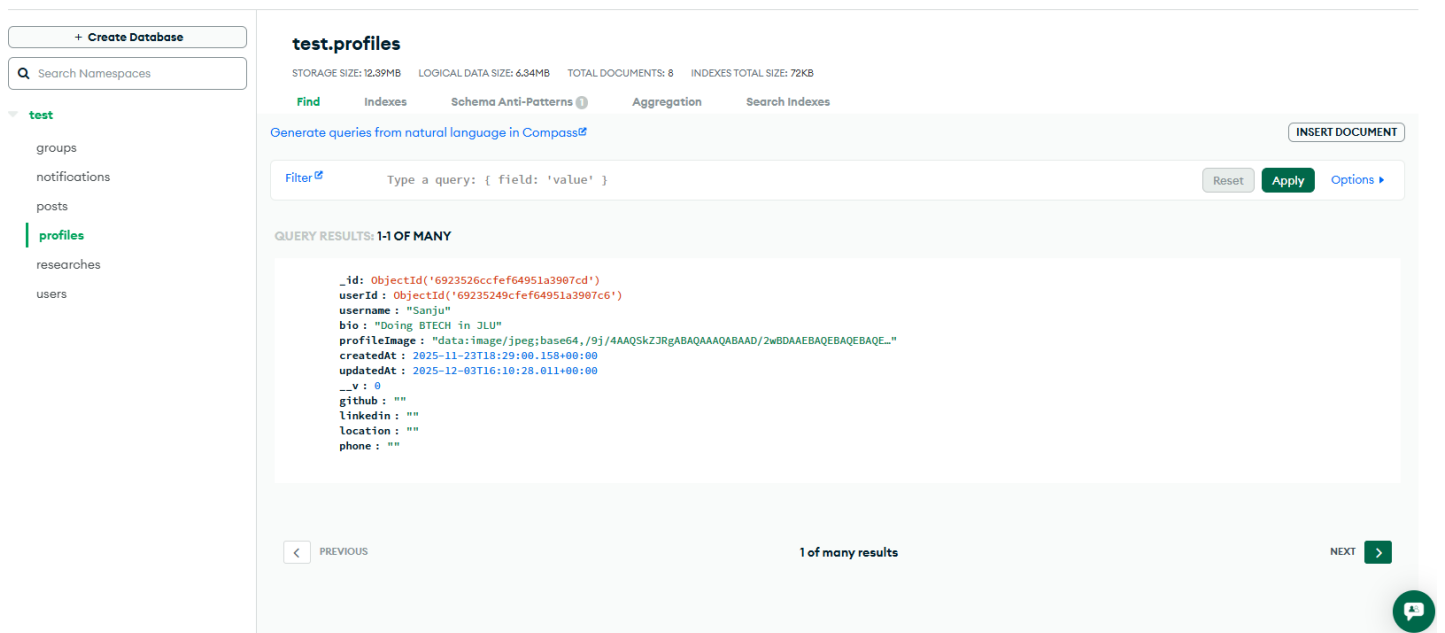


Figure 4.11: Profile Database

Description: In the images stored data of profiles are shown in the JSON format which consist key:value pairs.

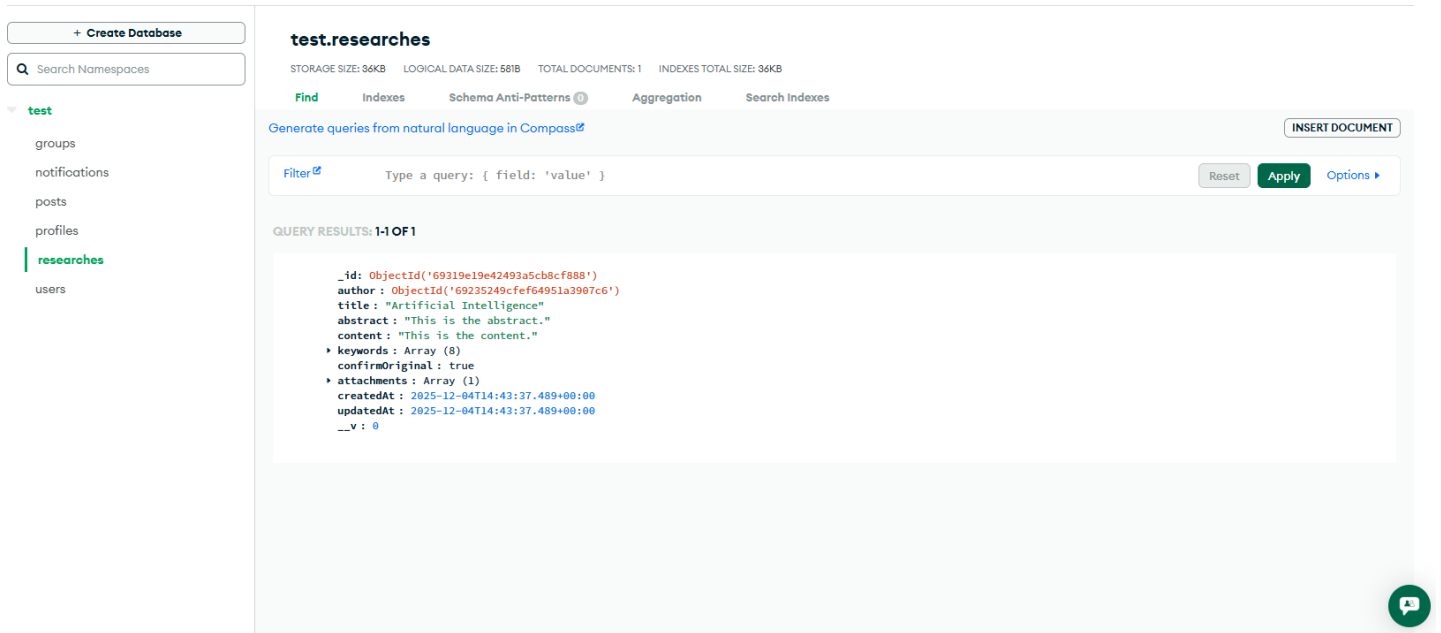


Figure 4.12: Research Database

Description: In the images stored data of researches are shown in the JSON format which consist key:value pairs.

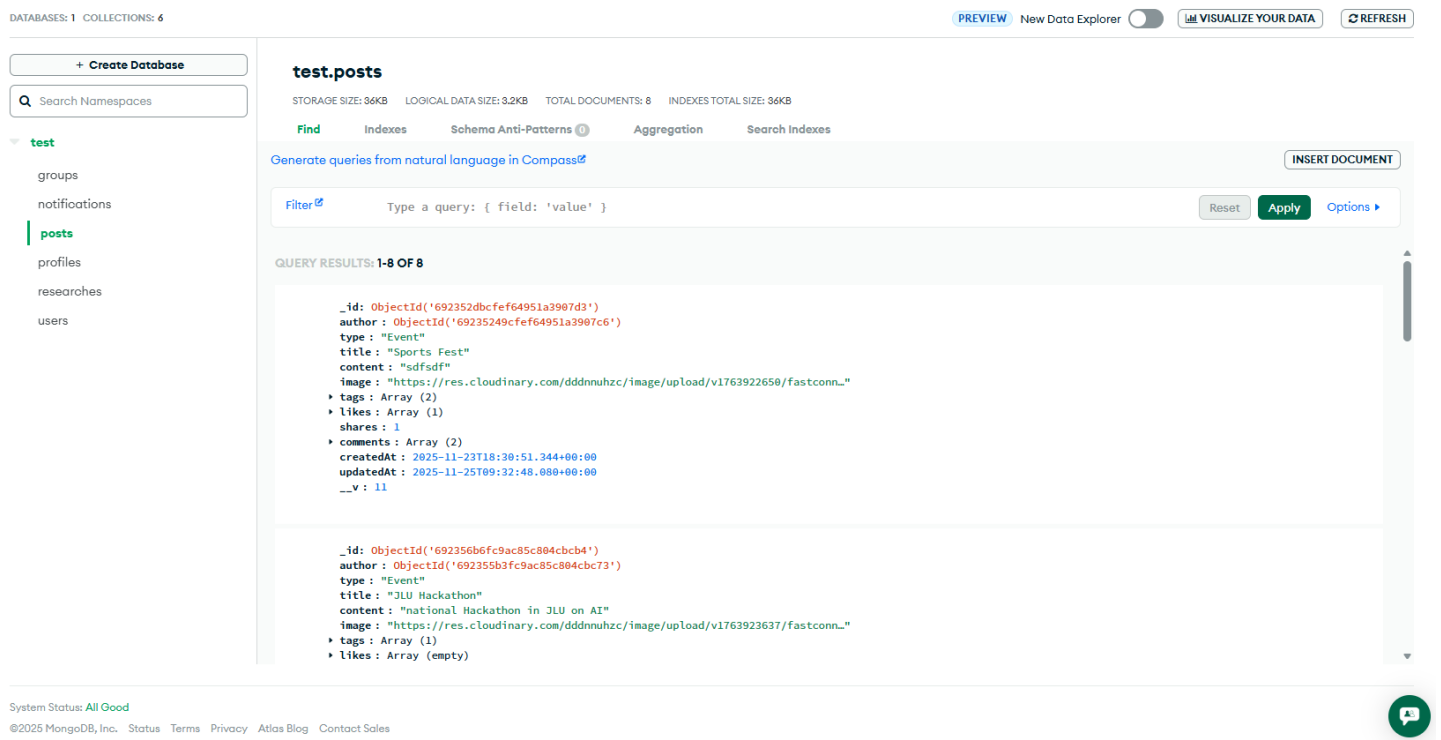


Figure 4.13: Post Database

Description: In the images stored data of posts are shown in the JSON format which consist key:value pairs.

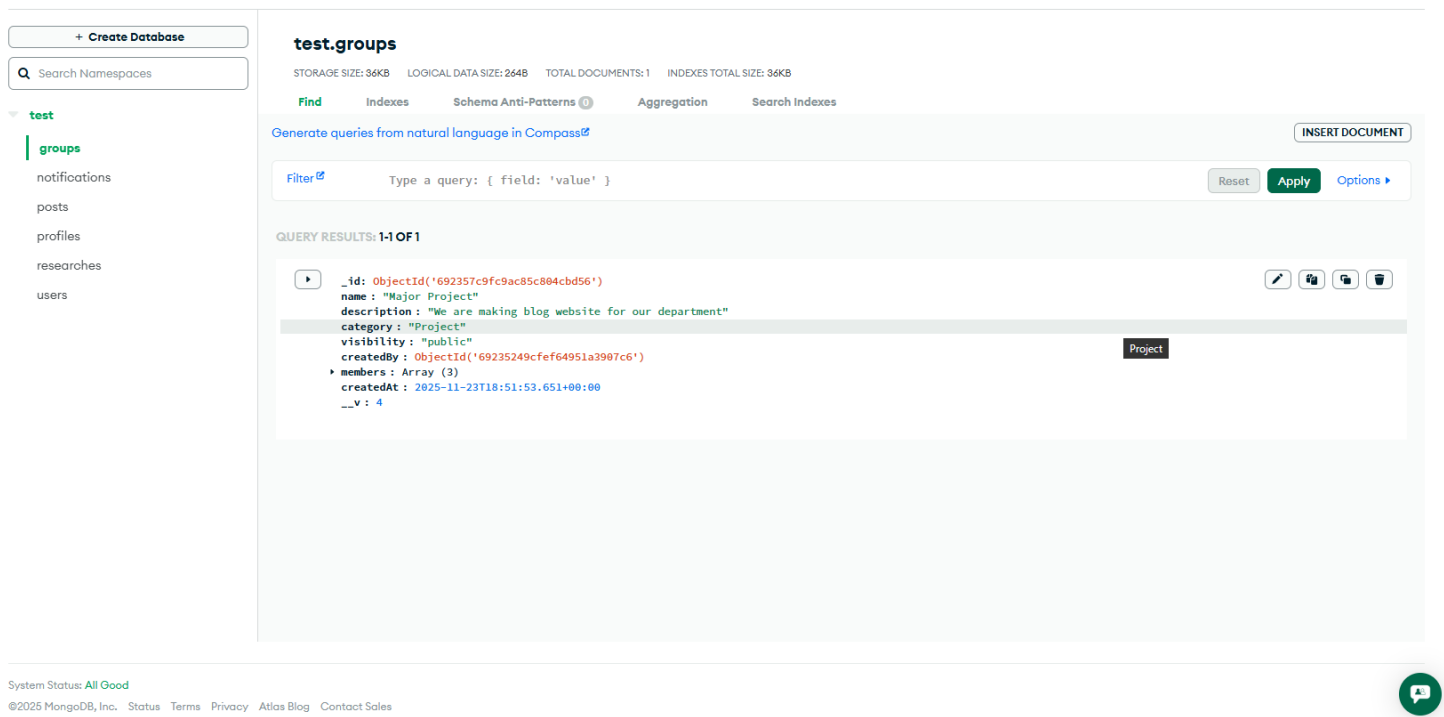


Figure 4.14: Groups Database

Description: In the images stored data of groups are shown in the JSON format which consist key:value pairs.

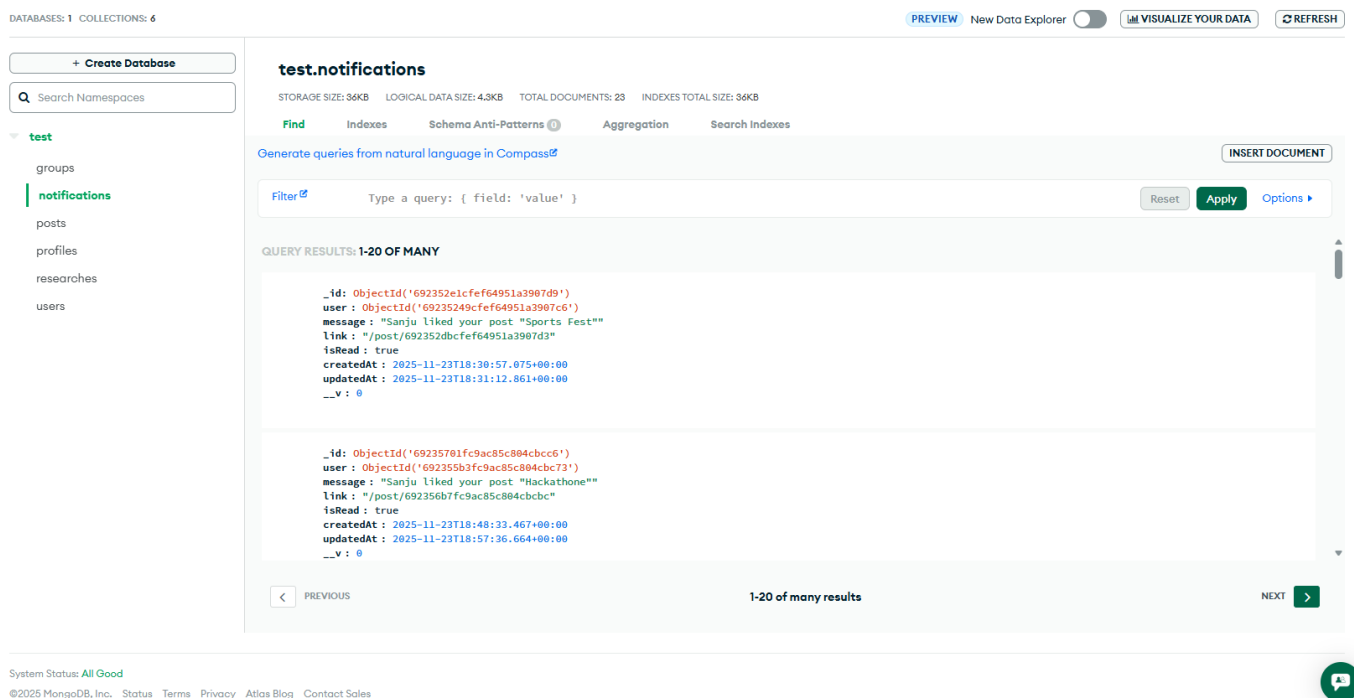


Figure 4.15: Notification Database

Description: In the images stored data of notifications are shown in the JSON format which consist key:value pairs.

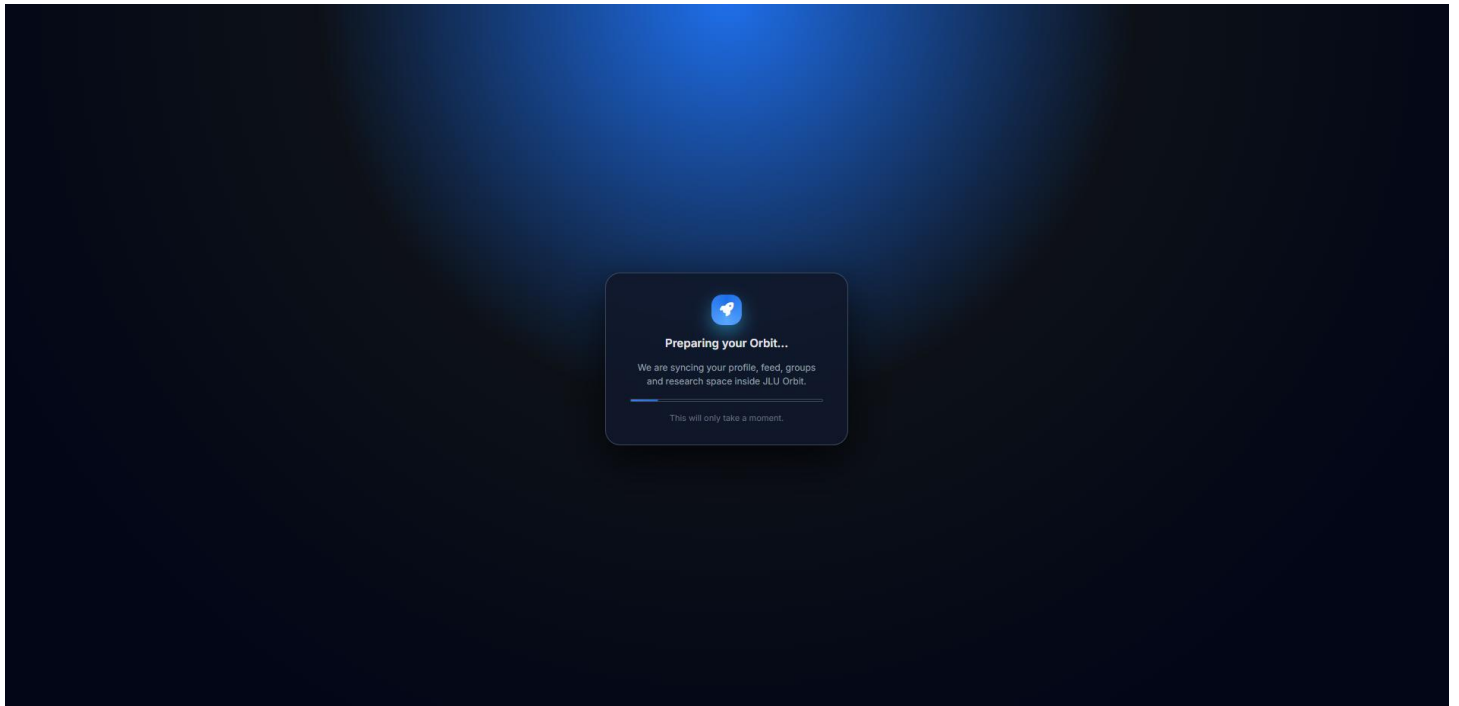


Figure 4.16: Loading Page

Description: This image contains our website's loading page after the successful login/signup by user has been done.

CHAPTER-5

CONCLUSIONS / RECOMMENDATIONS

5.1 Conclusion

JLU Orbit successfully achieves its core objective of creating a centralized, scalable, and user-friendly department-level blogging system. The project transitions the department's communication from a fragmented, informal set of tools to a formal, persistent, and professional platform.

By leveraging the modern stack of Node.js, Express.js, and MongoDB, the application demonstrates high performance, robust security, and technical maintainability. It fulfills all functional requirements, providing a secure dashboard for content creation and a dynamic public interface for information dissemination. This project serves as a strong foundation for a comprehensive digital communication strategy for the department.

5.2 Recommendations

While the current system is a fully functional MVP, it can be expanded with several key features to increase its utility:

1. **Role-Based Access Control (RBAC):** Implement a more granular permissions system to differentiate between "Admin" (can manage all posts) and "Faculty" (can only manage their own posts).
2. **Advanced Content Editor:** Replace the standard `<textarea>` with a Rich Text Editor (e.g., TinyMCE, Quill.js) to allow for text formatting, headings, and lists.
3. **Search and Filtering:** Add a search bar to allow users to find posts by keywords, and add filtering by category (e.g., "Events," "Academics," "Announcements") or date.
4. **Student Feedback System:** Integrate a moderated comments section for each post to allow for student questions and faculty-led discussion.
5. **Group Chat Module:** Add a real-time chat section inside group, enabling members to communicate instantly.
 - Built using Socket.io.
 - Supports text, attachments and read receipts.
 - Helps project teams, research groups, and clubs collaborate more effectively.

CHAPTER-6

REFERENCES

- M. Haverbeke, *Eloquent JavaScript*, 3rd ed. San Francisco, CA, USA: No Starch Press, 2018.
- A. Choudhury, *MongoDB: The Definitive Guide*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- Node.js Foundation, *Node.js v18.x Documentation*. 2025. [Online]. Available: <https://nodejs.org/docs/>
- Express.js Team, *Express.js 4.x API Reference*. 2025. [Online]. Available: <https://expressjs.com/>
- MongoDB, Inc., *Mongoose v6.7 Documentation*. 2025. [Online]. Available: <https://mongoosejs.com/docs/>
- Visual Studio Code, *Documentation*. 2025. [Online]. Available: <https://code.visualstudio.com/docs>
- Postman Learning Center, *Postman API Platform Documentation*. 2025. [Online]. Available: <https://learning.postman.com/>

CHAPTER-7

APPENDICES

Appendix A: GitHub Repository Link

The complete source code for this project is hosted on GitHub at the following private repository:

- [Project Github Link JLU Orbit](#)

Appendix B: API Endpoint Documentation

Method	Endpoint	Description	Auth Required
POST	/api/auth/login	Logs in a user and returns a JWT.	No
POST	/api/auth/register	Creates a new user (admin only).	Yes (Admin)
GET	/api/posts	Retrieves a list of all posts.	No
GET	/api/posts/:id	Retrieves a single post by its ID.	No
POST	/api/posts	Creates a new post.	Yes
PUT	/api/posts/:id	Updates an existing post by its ID.	Yes
DELETE	/api/posts/:id	Deletes a post by its ID.	Yes

Table 7.1: API Endpoint Documentation